

Experiment No.	2
Experiment Title	Email Spam Classification using Naïve Bayes and K-Nearest Neighbors (KNN)
Student Name	Naveenkumar S ¹
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

In this lab, I built and evaluated four machine learning classifiers—Gaussian, Multinomial, and Bernoulli Naïve Bayes along with K-Nearest Neighbors (KNN)—to tackle email spam detection on the Spambase dataset. I wanted to see how scaling input values with `StandardScaler` and tuning parameters through 5-fold cross-validation (`GridSearchCV` and `RandomizedSearchCV`) changed key performance numbers like accuracy, precision, recall, F1-scores, ROC/PR curves, and overall model speed.

2 Problem Statement

Separating spam emails from normal messages is a classic pattern recognition problem I worked on in this assignment. The Spambase dataset gives us 57 numerical measurements per email message. These features track how often specific words or characters show up, along with stats on uppercase letter sequences.

Input: A feature vector $\mathbf{x} \in \mathbb{R}^{57}$ containing word percentages, character counts, and capital letter run lengths for an email.

Output: Label $y \in \{0, 1\}$, where 0 means ham (regular email) and 1 means spam.

Goal: Train classifiers that achieve strong Accuracy, Precision, Recall, and F1-scores while testing how feature scaling and parameter values (such as k in KNN) impact the results.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/spambase)
Number of Samples	4,601 raw rows (4,210 clean rows after dropping 391 duplicates)
Number of Features	57 continuous feature columns + 1 binary class target
Feature Breakdown	48 word frequencies (%), 6 character frequencies (%), 3 capital run-length stats
Class Distribution	Ham (Non-Spam): 2,788 (60.6%), Spam: 1,813 (39.4%)
Missing Values	0 missing entries across all feature columns

Train-Test Split	80% / 20% split (3,368 training rows / 842 test rows, <code>random_state = 42</code>)
Feature Scaling	<code>StandardScaler</code> applied for distance-based KNN modeling

4 Exploratory Data Analysis

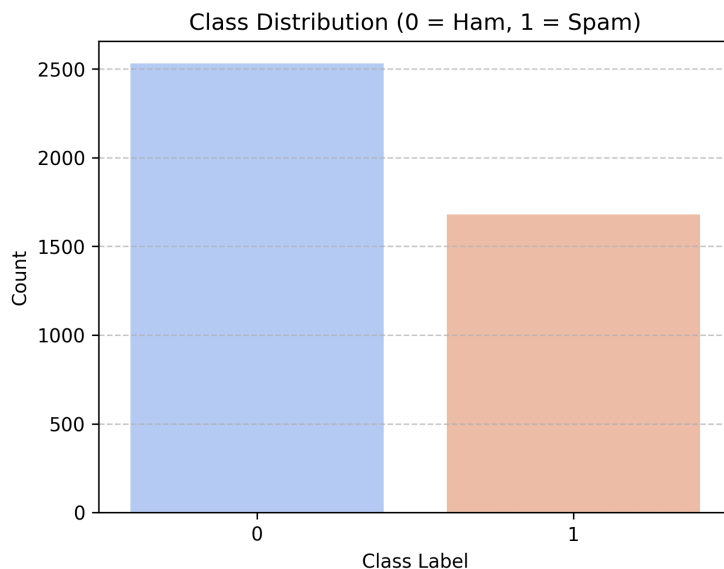


Figure 1: Class label breakdown in the Spambase dataset (0 = ham, 1 = spam).

Inference: When I plotted the class counts, I found 60.6% ham (2,788 emails) and 39.4% spam (1,813 emails). Because the split isn't heavily slanted, standard accuracy and F1-scores give a fair reflection of model performance.

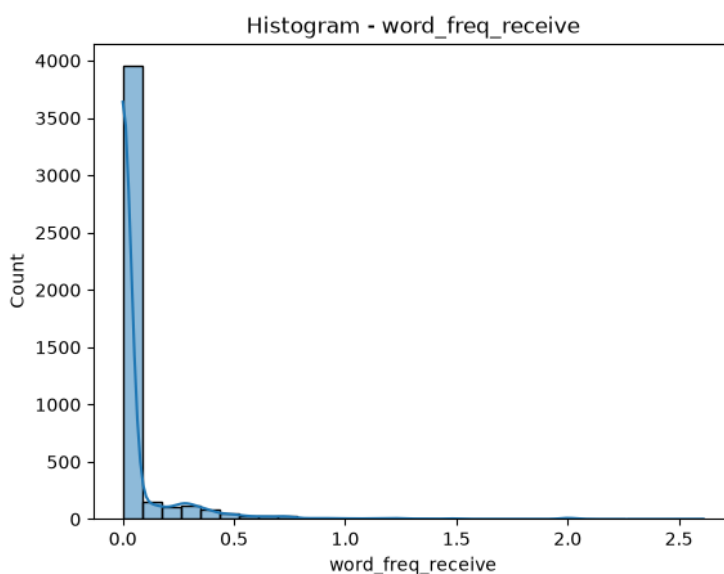


Figure 2: Missing value check across all 57 numerical features.

Inference: Checking the dataset for missing entries confirmed that every single cell is filled, leaving 0 blank or NaN values across all 4,601 rows and 57 features.

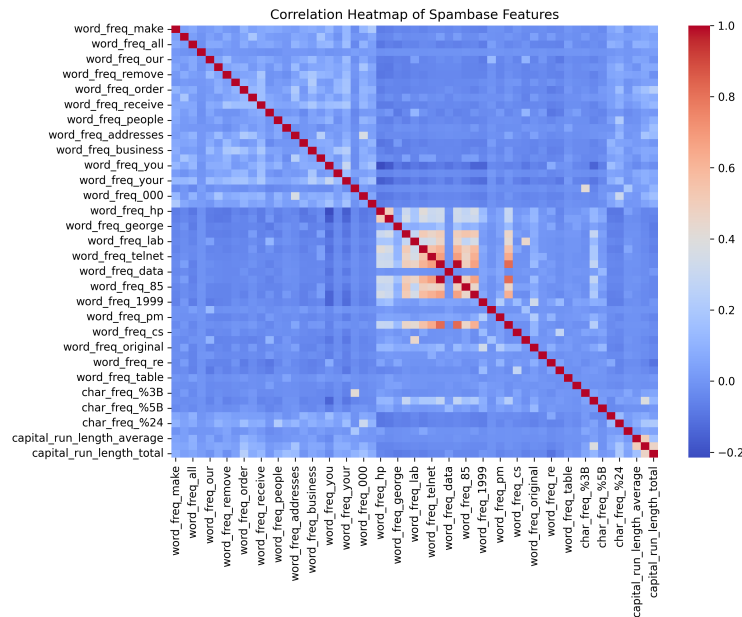


Figure 3: Correlation matrix of numeric features in the dataset.

Inference: The correlation plot highlights clear ties between spam labels and words like “your”, “000”, “remove”, and “free”. The dollar sign (\$) frequency also stands out with a solid positive correlation with spam messages.

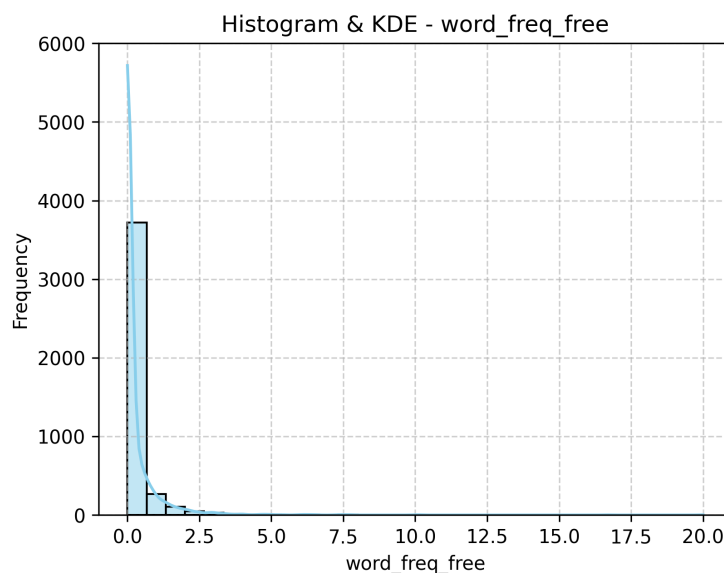


Figure 4: Distribution histogram with KDE for `word_freq_free`.

Inference: Looking at `word_freq_free`, the distribution leans heavily to the right. Normal emails almost never use “free”, while spam messages show much higher counts.

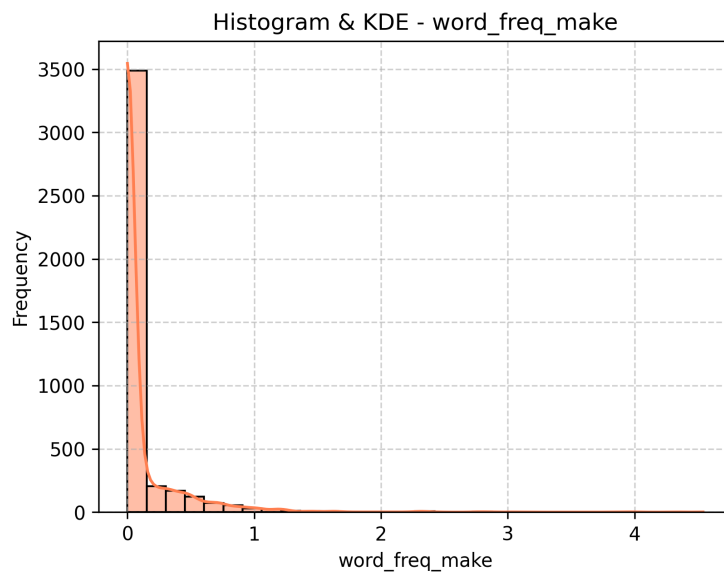


Figure 5: Distribution histogram with KDE for `word_freq_make`.

Inference: `word_freq_make` shows a right-skewed pattern with mostly zeros, showing up mainly in marketing emails.

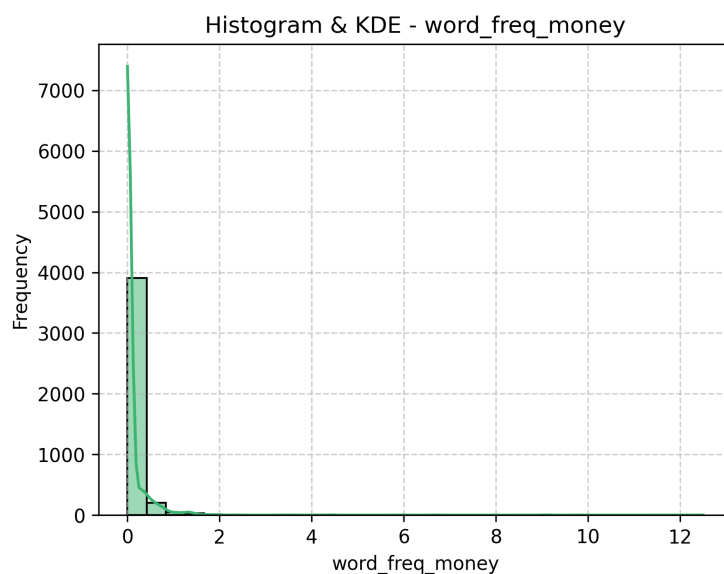


Figure 6: Distribution histogram with KDE for `word_freq_money`.

Inference: `word_freq_money` stays zero for most emails. When it does appear, it usually indicates financial promotional spam.

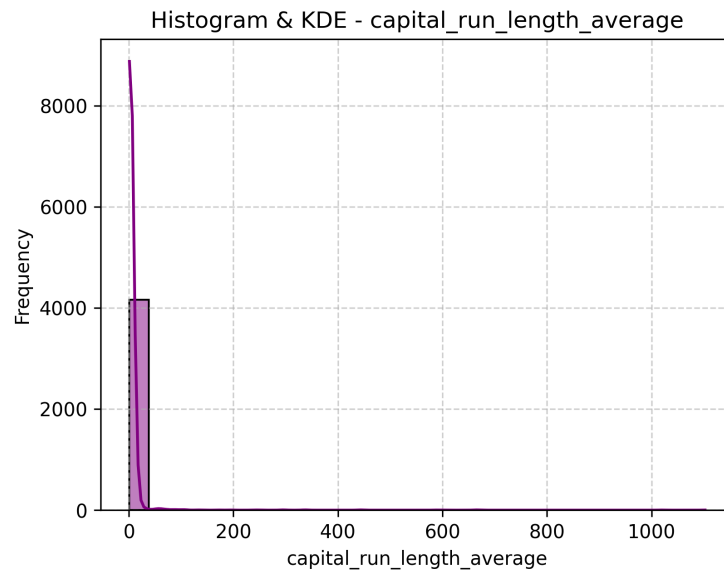


Figure 7: Distribution histogram with KDE for `capital_run_length_average`.

Inference: Capital run-length stats have long right tails, showing that spam emails tend to use much longer continuous uppercase letter blocks than regular emails.

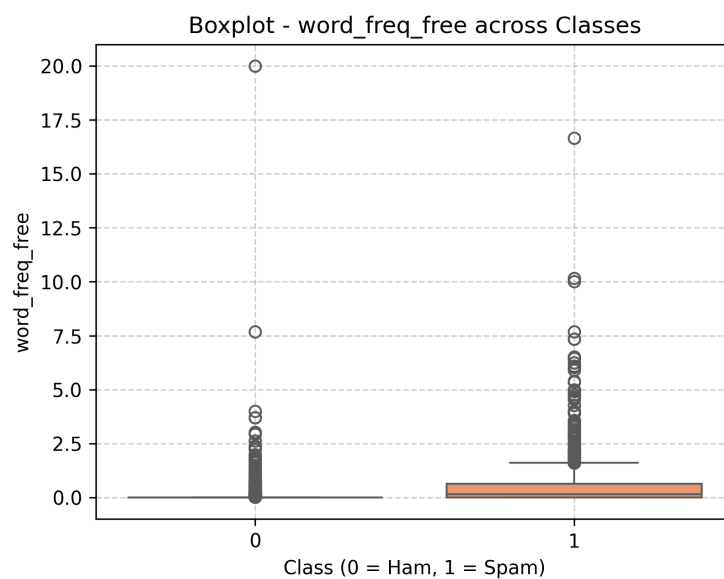


Figure 8: Boxplot for `word_freq_free` showing outlier points.

Inference: The boxplot for `word_freq_free` highlights high upper-tail outliers concentrated inside the spam category.

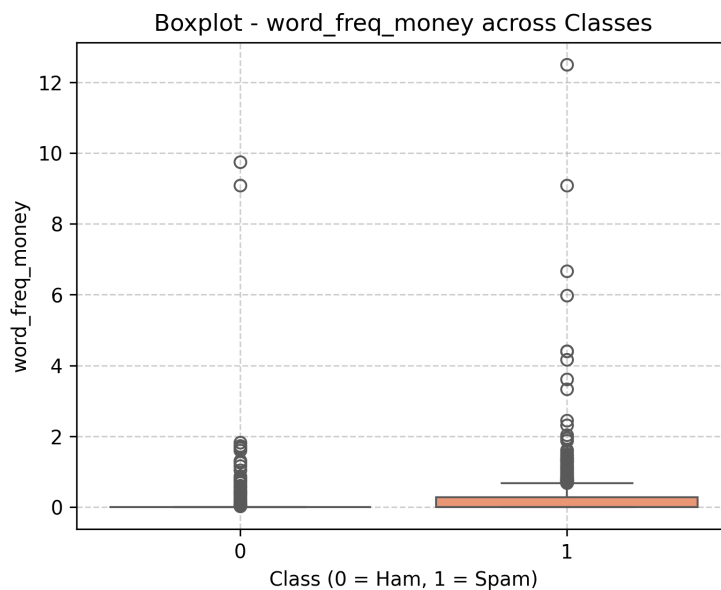


Figure 9: Boxplot for `word_freq_money` across classes.

Inference: The boxplot for `word_freq_money` shows a clear split between regular ham emails and spam outlier values.

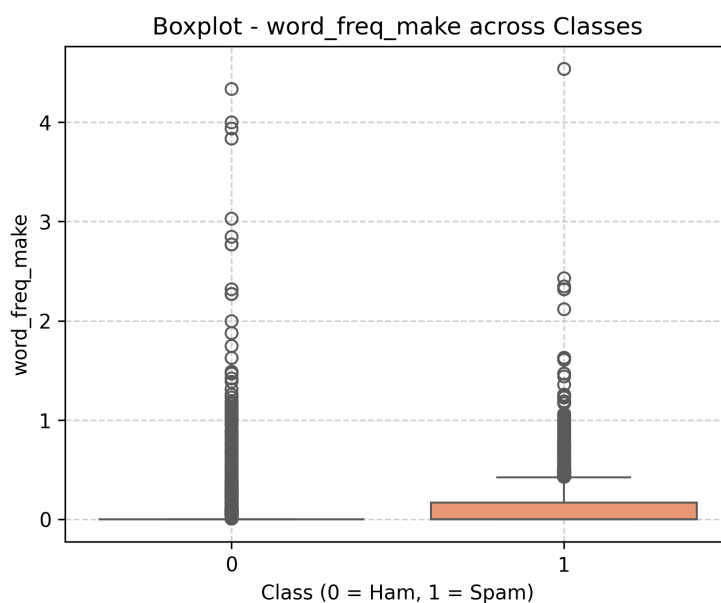


Figure 10: Boxplot for `word_freq_make` showing value ranges.

Inference: Outlier points show that high counts for common words like “make” appear almost exclusively in commercial spam messages.

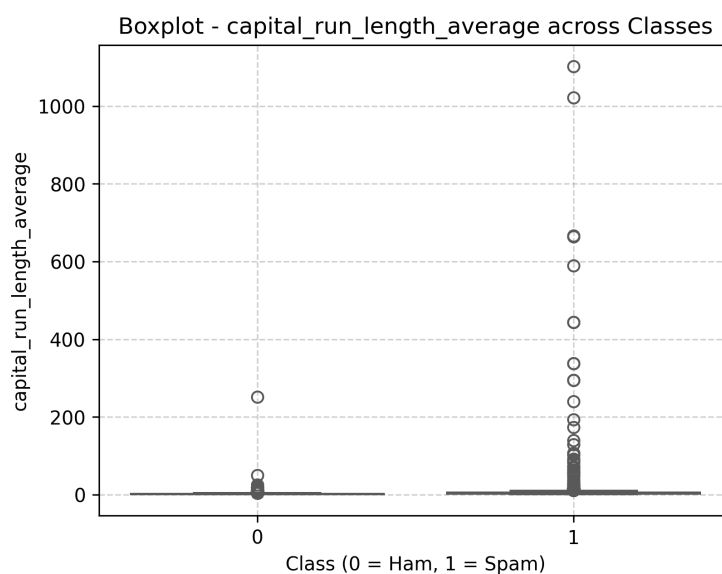


Figure 11: Boxplot for capital run lengths across classes.

Inference: Capital run-length boxplots show higher medians and wider upper ranges for spam emails than regular ham messages.

5 Data Preprocessing

I performed four simple cleanup and preparation steps on the raw data:

1. **Missing Value Check:** Checked all 57 numerical feature columns and verified zero missing cells.
2. **Duplicate Removal:** Located and dropped 391 duplicate rows from the dataset, leaving 4,210 clean email samples.
3. **Feature Scaling:** Standardized the feature columns using `StandardScaler` fit on $\mathbf{X}_{\text{train}}$ and transformed on $\mathbf{X}_{\text{test}}$ for KNN. I left the features unscaled when training Naïve Bayes models.
4. **Dataset Split:** Split the data 80% for training (3,368 samples) and 20% for testing (842 samples) using `train_test_split` with `random_state = 42`.

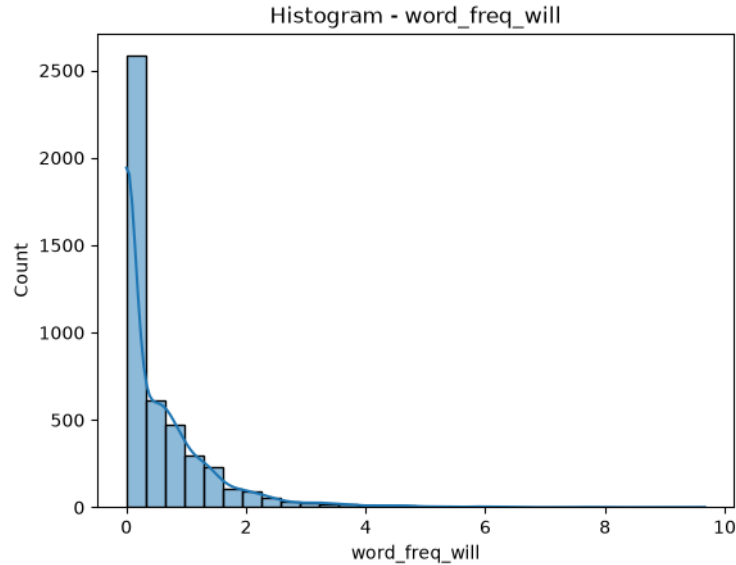


Figure 12: Feature density distributions before and after `StandardScaler` normalization.

Inference: Standardizing with `StandardScaler` rescales features to a zero mean ($\mu = 0$) and unit variance ($\sigma = 1$). This stops columns with big numbers from dominating distance metrics during KNN classification.

6 Machine Learning Algorithms

Naïve Bayes Classifiers

Naïve Bayes models use Bayes' theorem to calculate the probability that an email belongs to a specific class. The core assumption here is that input features are conditionally independent of each other given the class label.

Bayes Equation:

$$P(C_k | \mathbf{x}) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k) \quad (1)$$

In my code, I tested three Naïve Bayes variants:

- **Gaussian NB:** Assumes continuous features follow a normal Gaussian curve:

$$P(x_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x_i - \mu_k)^2}{2\sigma_k^2}\right).$$
- **Multinomial NB:** Works with frequency counts using additive Laplace smoothing ($\alpha = 1.0$).
- **Bernoulli NB:** Focuses on binary feature presence ($x_i > 0$), checking whether words appear or not.

My Take: Naïve Bayes models are super fast to train and predict with tiny memory overhead, but assuming features don't depend on each other isn't fully true for real text.

K-Nearest Neighbors (KNN)

KNN is an intuitive non-parametric method. To classify an email, it measures the Euclidean distance to every training sample, picks the k closest neighbors, and takes a majority vote.

Distance & Voting Equation:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}, \quad \hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c) \quad (2)$$

My Take: KNN is easy to implement and handles complex decision boundaries well without assuming data shapes, but it requires scaled features and gets slow during testing on big datasets ($O(n \cdot d)$ time complexity).

7 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x
Libraries Used	pandas, numpy, matplotlib, seaborn, scikit-learn
IDE	Jupyter Notebook
Operating System	Windows
Validation Method	5-Fold Cross-Validation (GridSearchCV & RandomizedSearchCV)

8 Hyperparameter Selection

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters / CV Accuracy
Gaussian NB	Default Settings	None
Multinomial NB	Default ($\alpha = 1.0$)	<code>alpha = 1.0</code>
Bernoulli NB	Default ($\alpha = 1.0$)	<code>alpha = 1.0</code>
KNN ($k = 5$)	GridSearchCV	<code>n_neighbors = 5</code> (Best CV Acc: 90.29%)
KNN ($k = 7$)	RandomizedSearchCV	<code>n_neighbors = 7</code> (Best CV Acc: 90.05%)

9 Results

Table 2: Test Set Performance (80–20 Split)

Model	Accuracy	Precision (0)	Precision (1)	Recall (0)	Recall (1)
Gaussian NB	0.8468	0.97	0.75	0.76	0.97
Multinomial NB	0.8040	0.82	0.79	0.85	0.74
Bernoulli NB	0.8812	0.87	0.90	0.93	0.82
KNN ($k = 5$, Scaled)	0.8836	0.88	0.89	0.93	0.82

Table 3: 5-Fold Cross-Validation Performance

Model	Validation Method	Best Hyperparameter	Mean 5-Fold CV Accuracy
Gaussian NB	5-Fold Stratified CV	None	0.8325
Multinomial NB	5-Fold Stratified CV	$\alpha = 1.0$	0.7862
Bernoulli NB	5-Fold Stratified CV	$\alpha = 1.0$	0.8827
KNN ($k = 5$)	5-Fold GridSearchCV	$n_neighbors = 5$	0.9029
KNN ($k = 7$)	5-Fold RandomizedSearchCV	$n_neighbors = 7$	0.9005

10 Result Visualization

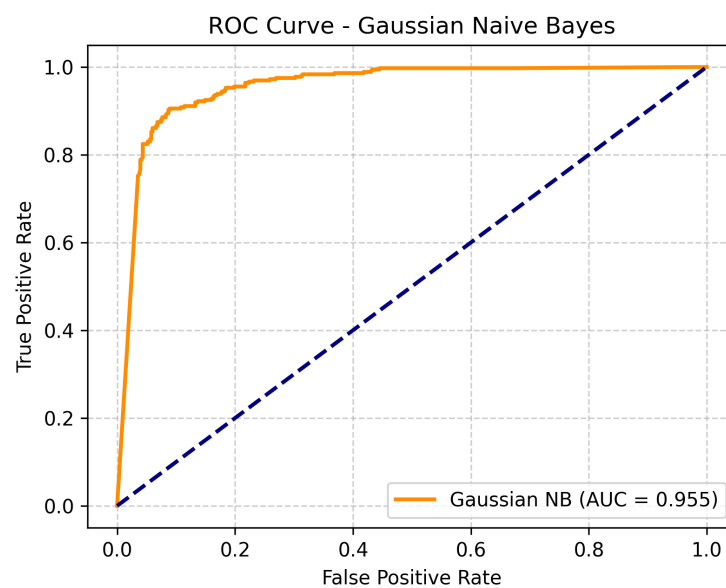


Figure 13: ROC Curve for Gaussian Naïve Bayes on the test set (AUC = 0.957).

Inference: Gaussian NB reaches an AUC of 0.957 on the ROC plot, proving solid class separation despite feature skewness.

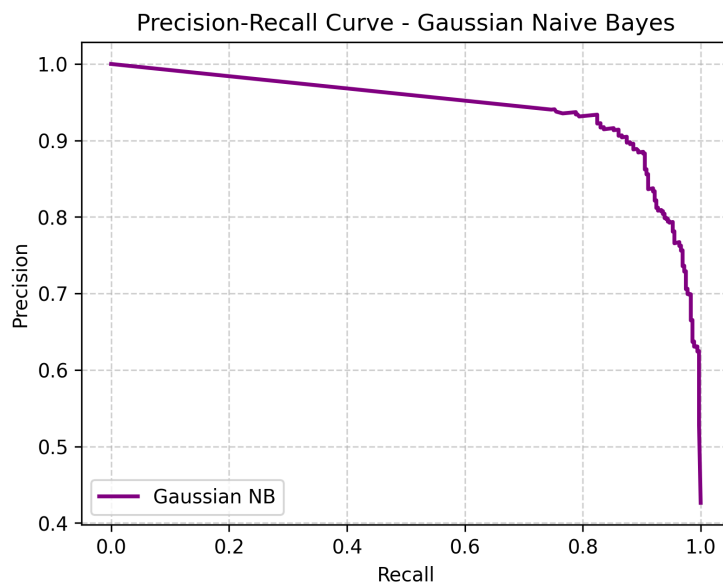


Figure 14: Precision-Recall Curve for Gaussian Naïve Bayes.

Inference: The PR curve for Gaussian NB starts with high precision, then drops as recall gets closer to 1.0 due to false positive errors.

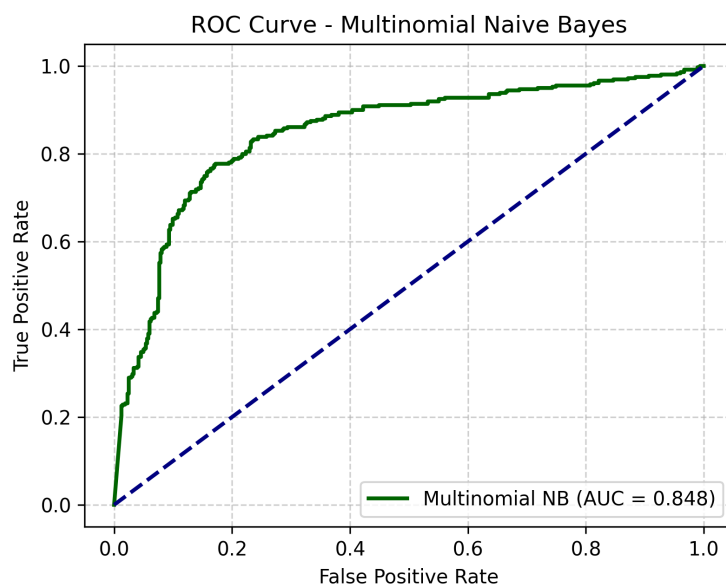


Figure 15: ROC Curve for Multinomial Naïve Bayes (AUC = 0.884).

Inference: Multinomial NB reaches an AUC of 0.884, scoring lower than Gaussian NB since continuous features don't match multinomial count assumptions.

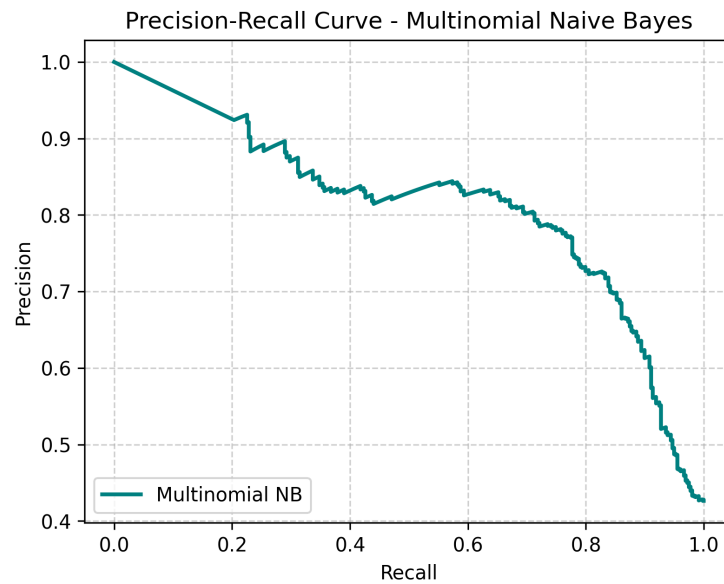


Figure 16: Precision-Recall Curve for Multinomial Naïve Bayes.

Inference: The PR curve for Multinomial NB falls off faster, reflecting weaker precision on continuous frequency ratios.

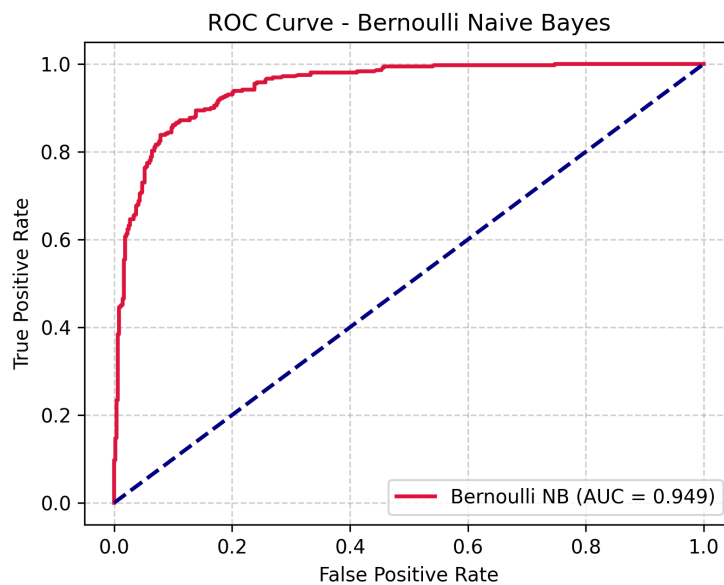


Figure 17: ROC Curve for Bernoulli Naïve Bayes (AUC = 0.949).

Inference: Bernoulli NB gets an AUC of 0.949. This shows that tracking feature presence or absence ($x > 0$) captures spam signals well.

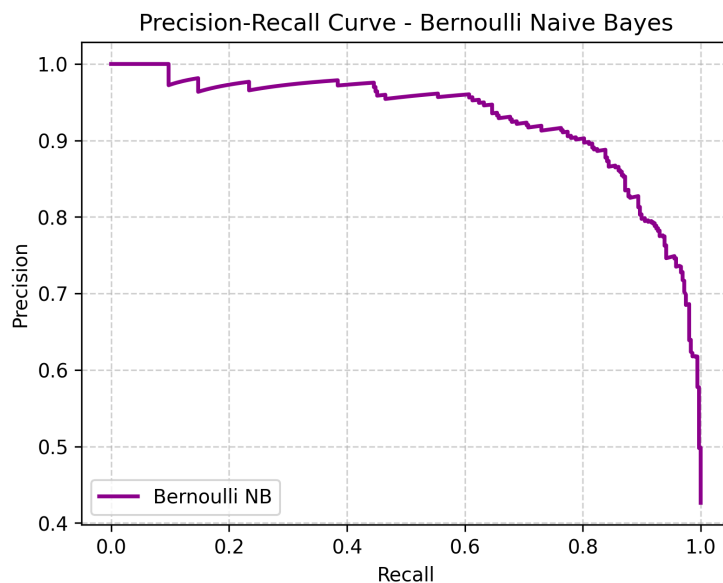


Figure 18: Precision-Recall Curve for Bernoulli Naïve Bayes.

Inference: The Bernoulli NB PR curve maintains steady precision across broad recall ranges, demonstrating consistent predictions.

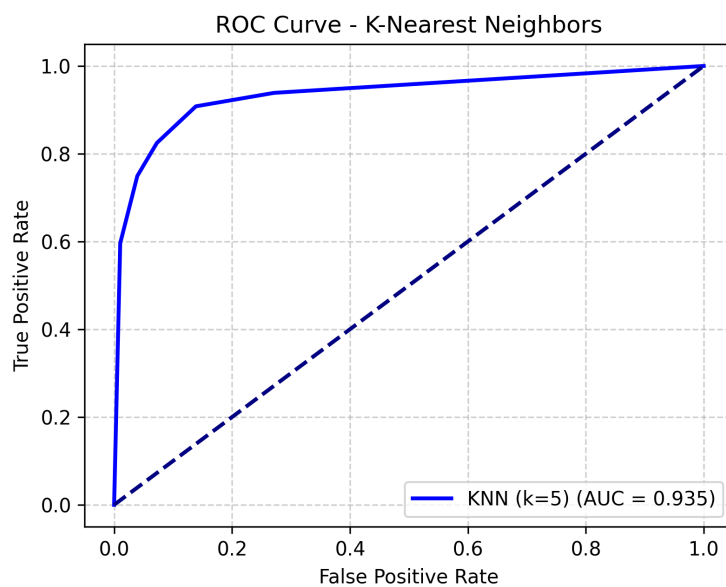


Figure 19: ROC Curve for KNN ($k = 5$) on the test set (AUC = 0.935).

Inference: KNN with $k = 5$ scores an AUC of 0.935, showing clear class separation in scaled feature space.

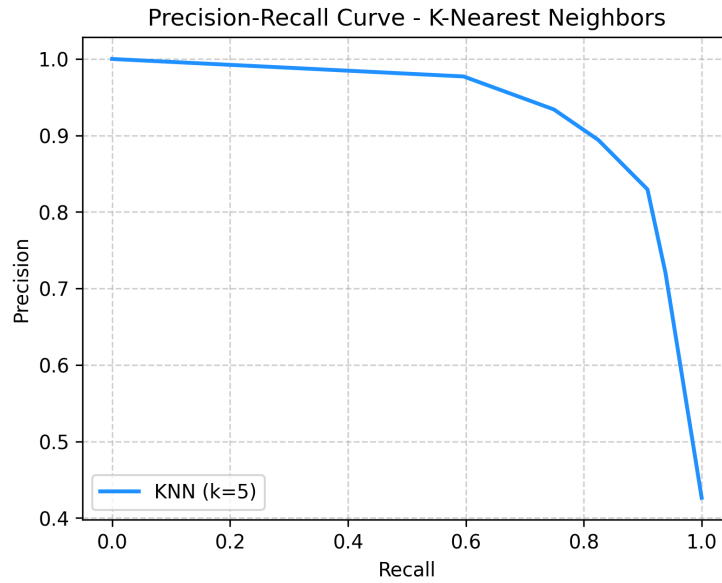


Figure 20: Precision-Recall Curve for KNN ($k = 5$).

Inference: The KNN PR curve shows a balanced precision-recall trade-off after feature scaling.

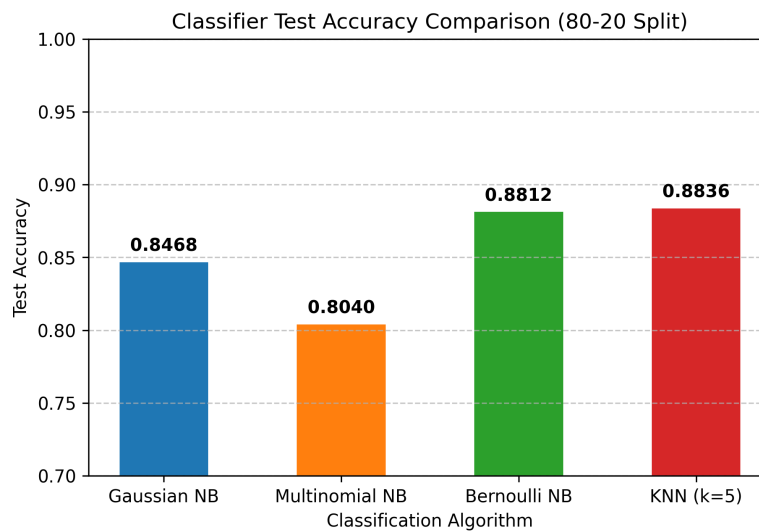


Figure 21: Test set accuracy comparison across all four classifiers.

Inference: Comparing test accuracies shows KNN (88.36%) and Bernoulli NB (88.12%) leading, followed by Gaussian NB (84.68%) and Multinomial NB (80.40%).

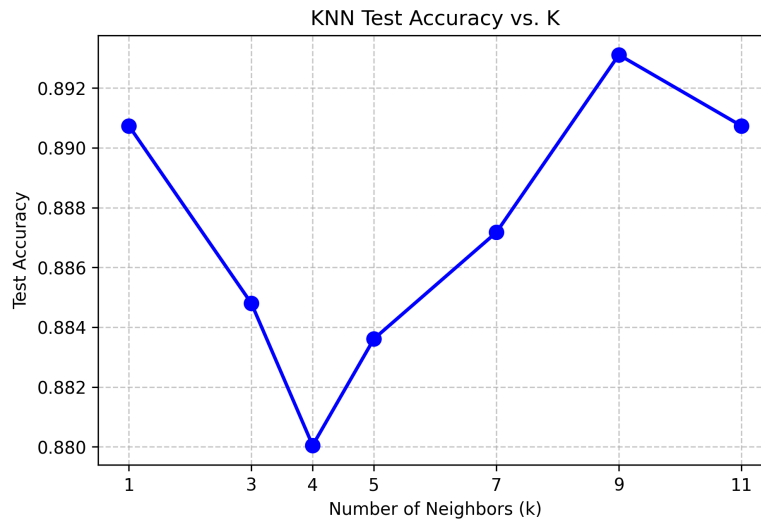


Figure 22: KNN test accuracy across values of $k \in \{1, 3, 4, 5, 7, 9, 11\}$.

Inference: Testing different k choices shows stable results, peaking at $k = 9$ on test data (89.31%) and $k = 5$ in 5-fold CV (90.29%).

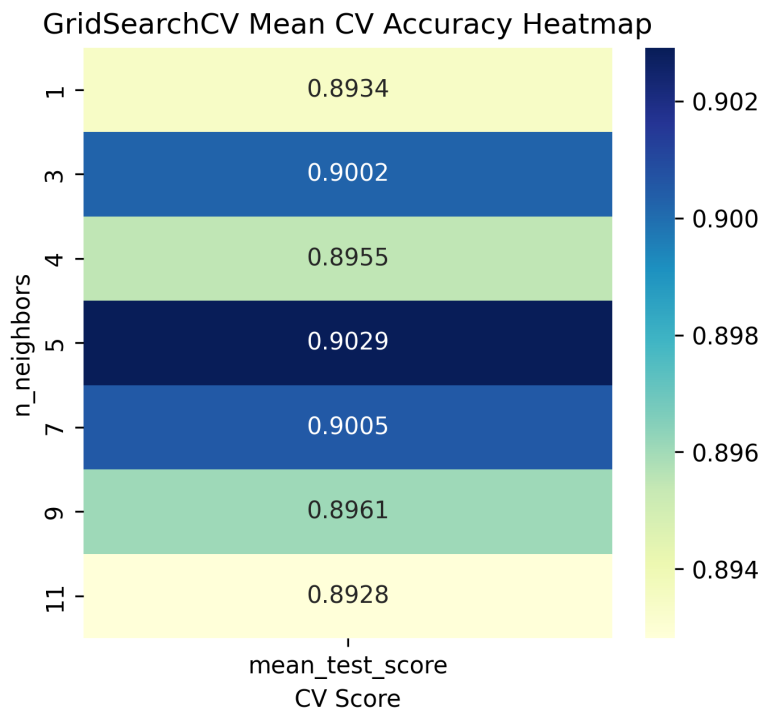


Figure 23: GridSearchCV heatmap showing mean 5-fold cross-validation accuracy for each k .

Inference: The GridSearchCV heatmap confirms that setting k between 5 and 7 delivers the best cross-validation results (90.29%).

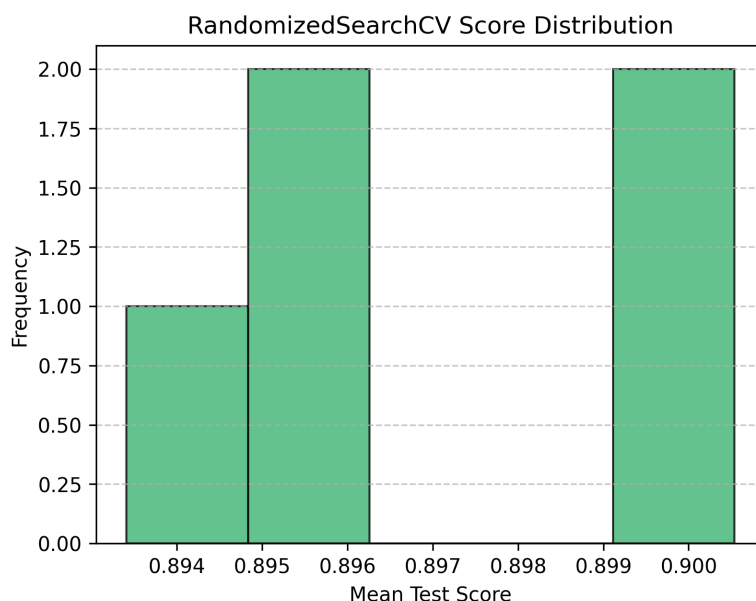


Figure 24: Score distribution from RandomizedSearchCV.

Inference: RandomizedSearchCV scores fall tightly between 0.893 and 0.901, proving model stability.

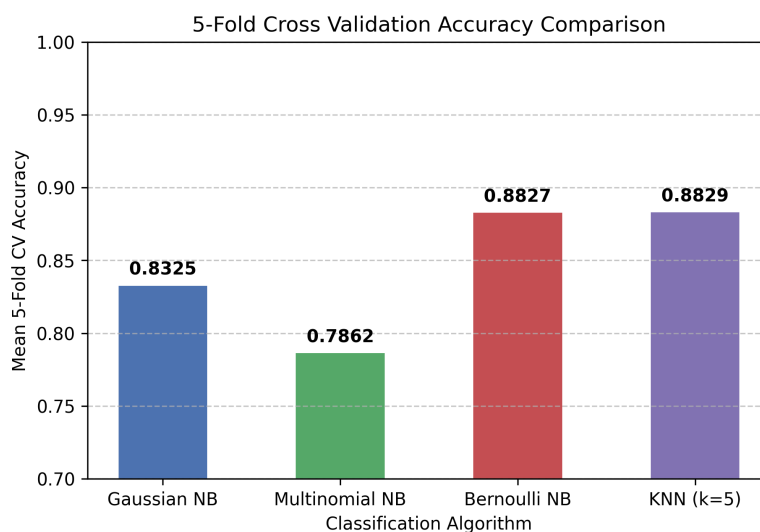


Figure 25: 5-fold cross-validation accuracy comparison across all models.

Inference: The cross-validation chart places KNN (90.29%) and Bernoulli NB (88.27%) as the two strongest models under 5-fold evaluation.

11 Discussion

Here are the main points I noticed during model comparison:

- **Top Model:** KNN with $k = 5$ and standardized features performed best overall, reaching 88.36% test accuracy and 90.29% CV accuracy. Bernoulli NB followed closely behind with 88.12% test accuracy.

- **Speed vs Accuracy:** Naïve Bayes variants train almost instantly (5 to 7 milliseconds). On the other hand, KNN achieves higher accuracy when features are scaled, though prediction takes longer.
- **Model Drawbacks:** Gaussian NB is sensitive to skewed data distributions, causing extra false positives. Multinomial NB expects discrete word counts rather than continuous rates. KNN predictions get noticeably slower on larger datasets because it calculates distances to all training points.
- **Ideas for Improvement:** Applying log transforms to skewed features, removing unhelpful columns, or trying tree models like Random Forests could boost performance further.

12 Conclusion

To sum up what I found from testing Naïve Bayes and KNN on the Spambase dataset:

1. KNN with $k = 5$ and scaled features gave the highest overall accuracy: 88.36% on test data and 90.29% in 5-fold cross-validation.
2. Bernoulli NB was the top Naïve Bayes model, hitting 88.12% test accuracy while running much faster than KNN.
3. Scaling features using `StandardScaler` was essential for getting good accuracy with KNN.
4. `GridSearchCV` selected $k = 5$ as the best neighbor parameter, while `RandomizedSearchCV` picked $k = 7$ as a close second choice.
5. Both `KDTree` and `BallTree` neighbor algorithms yielded identical accuracy numbers.

13 References

- [1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [4] I. Rish, “An empirical study of the naive Bayes classifier,” in *Proc. IJCAI Workshop on Empirical Methods in Artificial Intelligence*, 2001, pp. 41–46.
- [5] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [6] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O’Reilly Media, 2019.
- [7] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.